Reinforcement Learning

Clase 09

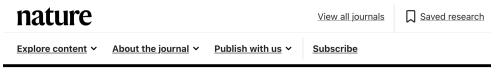
Ignacio Brevis

ignaciobrve@unisabana.edu.co

Universidad de La Sabana

14 octubre 2026





nature > news > article

NEWS | 07 October 2026 | Update 09 October 2026 | Clarification 12 October 2026

OpenAI posts 700 maths preprints online: mathematicians are up in arms

The firm used an unreleased model to tackle hundreds of high-profile problems.

By Davide Castelvecchi

Save article

"The research community is in uproar after OpenAI released a trove of more than 700 mathematical preprints entirely generated by artificial intelligence on 6 October. The ChatGPT maker [...] posted the preprints on the software repository Github"

README | Apache-2.0 license

Readme

This repository contains mathematical manuscripts and supporting proof artifacts produced by an internal OpenAI model.

As part of model development, we evaluate our models on open research problems. We expanded these evaluations after performance on our existing mathematical evaluations saturated. Some outputs build upon earlier results produced by the models.

This collection includes results at different stages of verification. Not all have accompanying Lean formalizations. We will continue to update this repository with Lean formalizations as we obtain them.

Some of the unformalized results could have issues. We will endeavor to fix any such issues quickly. We are also exploring community-hosted repositories for these materials.

Navigating the collection

The current catalogue contains 719 manuscripts organized into 372 families. A family groups related papers, which may include a principal result, companion arguments, consequences, or alternative proofs. Each family is classified by mathematical discipline.

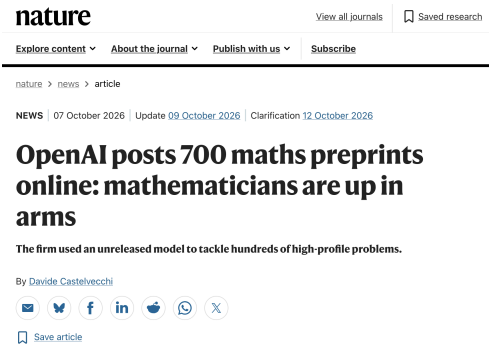
- Start with the overview for descriptions of the families.
- Use the manuscript map to find individual papers and their supporting materials.
- The `preprints/` directory contains PDFs, source files, and manuscript-specific citation and build instructions.
- The Lean library and formalization catalogue describe the available formal proofs, their associated papers, and verification configurations. See the Comparator instructions for additional checking instructions. The repository has ~42% top-line results formalized.
- Updates to the repo are described in the history.

https://www.nature.com/articles/d41586-026-03196-8 and https://github.com/openai/math

Levent Alpöge, a mathematician at rival company Anthropic, also in San Francisco, wrote on X that there will be “sad stories” for some researchers who might find their projects pre-empted by the OpenAI release. “We should put that aside for today though. It’s obviously the most significant moment in mathematical history,” he wrote, because it shows how quickly the technology has become useful to mathematicians.

Others say that a better approach would have been to collaborate with members of the mathematics community. Ursula Martin, a mathematician at the University of Oxford, UK, says that the publication of the preprints is instead “tossing the community a messy first draft and expecting us to clean it up for them”.

“I can’t bring myself to spend my time studying the outcome of this process, which I consider illegitimate”, says Michael Harris, a number theorist at Columbia University in New York City. “So I will not be reading these papers”.


nature

View all journals | Saved research

Explore content | About the journal | Publish with us | Subscribe

nature › news › article

NEWS | 07 October 2026 | Update 09 October 2026 | Clarification 12 October 2026

OpenAI posts 700 maths preprints online: mathematicians are up in arms

The firm used an unreleased model to tackle hundreds of high-profile problems.

By Davide Castelvecchi

Save article

https://www.nature.com/articles/d41586-026-03196-8

Repaso: SARSA & n-step SARSA

SARSA estima valores de acciones para una política dada, por lo que necesita de un paso de mejora de políticas para encontrar políticas óptimas.

SARSA:

necesita: $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$

para calcular:

TD

$$\begin{cases} q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)\Big[q_t(s_t, a_t) - \big(r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})\big)\Big], \\ q_{t+1}(s, a) = q_t(s, a), \quad \forall\, (s, a) \neq (s_t, a_t) \end{cases}$$

---

n-step SARSA:

necesita: $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, \ldots, r_{t+n}, s_{t+n}, a_{t+n})$

para calcular:

TD

$$\begin{cases} q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)\Big[q_t(s_t, a_t) - \big(r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n q_t(s_{t+n}, a_{t+n})\big)\Big], \\ q_{t+1}(s, a) = q_t(s, a), \quad \forall\, (s, a) \neq (s_t, a_t) \end{cases}$$

Q-learning busca estimar directamente acciones y políticas óptimas!

En vez de resolver la ecuación de Bellman:

$$q_\pi(s,a) = \mathbb{E}[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) | S_t = s, A_t = a]$$

Resolveremos la BOE en términos del valor de la acción:

$$q(s,a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} q(S_{t+1}, a') | S_t = s, A_t = a]$$

El algoritmo de Q-learning viene dado por: $(s_t, a_t, r_{t+1}, s_{t+1})$

TD

$$\begin{cases} q_{t+1}(s_t,a_t) = q_t(s_t,a_t) - \alpha(s_t,a_t)\Big[q_t(s_t,a_t) - \big(r_{t+1} + \gamma \max_{a'} q_t(s_{t+1},a')\big)\Big], \\ q_{t+1}(s,a) = q_t(s,a), \quad \forall\,(s,a) \neq (s_t,a_t) \end{cases}$$

Repaso: On policy / off policy learning

Algunos conceptos importantes:

- **Behavior policy**: política usada para generar las muestras/experiencias (política exploratoria).
- **Target policy**: política que se actualiza constantemente para converger a una política óptima (política de explotación).
- **On-policy learning**: cuando la behavior policy es la misma que la target policy.
- **Off-policy learning**: behavior policy y target policy son distintas.

Repaso

Se pueden escribir de forma general como: $q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha(s_t, a_t)\Big[q_t(s_t, a_t) - \bar{q}_t\Big]$

Método	**TD target**
Sarsa	$\bar{q}_t = r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})$
n-step Sarsa	$\bar{q}_t = r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n q_t(s_{t+n}, a_{t+n})$
Q-learning	$\bar{q}_t = r_{t+1} + \gamma \max_a q_t(s_{t+1}, a)$
Monte Carlo ($\alpha = 1$)	$\bar{q}_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots$

donde

Método	**Problema que resuleven**
Sarsa	$q(s,a) = \mathbb{E}\left[R_{t+1} + \gamma q(S_{t+1}, A_{t+1}) \mid S_t = s,\, A_t = a\right]$
n-step Sarsa	$q(s,a) = \mathbb{E}\left[R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^n q(S_{t+n}, A_{t+n}) \mid S_t = s,\, A_t = a\right]$
Q-learning	$q(s,a) = \mathbb{E}\left[R_{t+1} + \gamma \max_{a'} q(S_{t+1}, a') \;\middle|\; S_t = s,\, A_t = a\right]$
Monte Carlo	$q(s,a) = \mathbb{E}\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \mid S_t = s,\, A_t = a\right]$

Más ejemplos:

- Ver código `Example_02_optimal_policy_Q-learning_FrozenLake` y completar la función `q_learning` para obtener la política óptima.
- Ver código `Example_03_optimal_policy_Q-learning_Taxi.ipynb` y completar la función `q_learning` para obtener la política óptima.

Nuevo material

- Monte Carlo
- Sarsa
- n-step Sarsa
- Q-learning

Universidad de La Sabana

Métodos tabulares

Reinforcement Learning, segundo semestre

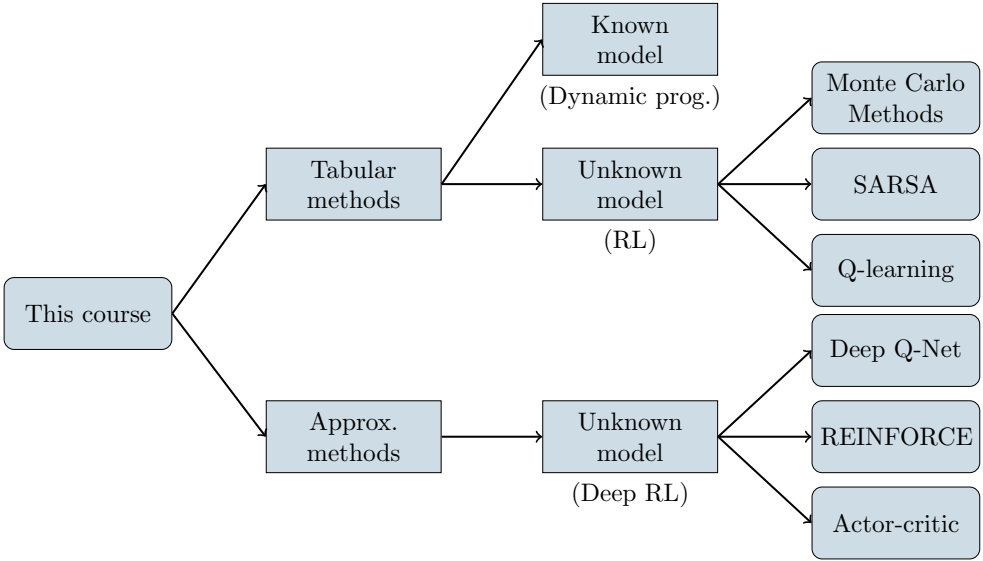
Ignacio Brevis

Versión 1.0. Si encuentras algún error o tienes comentarios, escríbeme a `ignaciobrve@unisabana.edu.co`.

Resumen

Esta guía trata el caso en el que el modelo **no se conoce** y el agente sólo dispone de muestras obtenidas al interactuar con el entorno. Presentamos cuatro algoritmos: **Monte Carlo**, que promedia retornos completos; **Sarsa**, que sustituye el retorno por una estimación *bootstrap*; **n-step Sarsa**, que interpola entre los dos anteriores; y **Q-learning**, que resuelve directamente la ecuación de optimalidad de Bellman. Al final se incluyen 10 ejercicios con pistas y soluciones completas.

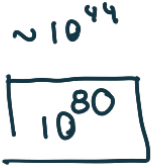
Contenido


This course
Tabular methods
Approx. methods
Known model
(Dynamic prog.)
Unknown model
(RL)
Unknown model
(Deep RL)
Monte Carlo Methods
SARSA
Q-learning
Deep Q-Net
REINFORCE
Actor-critic

Tabular vs approx. methods

Métodos tabulares funcionan bien cuando

- Problemas de Gridworld (académicos)
- Juegos de mesa con "razonable" número de estados
- MDP con cientos o (unos pocos) miles de estados
- Problemas donde los estados se pueden enumerar explícitamente



Métodos aproximados entran en juego cuando:

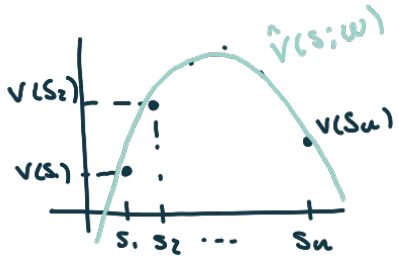
- Espacios de estados continuos (articulaciones de robots, velocidades, lectura de sensores)
- Inputs de alta dimensión (juegos de Atari con 84×84 píxeles)
- Simulaciones físicas complejas (conducción autónoma)
- Espacios discretos muy grandes (Go, con $\sim 10^{170}$ posibles tableros)

$256^{7056} \approx 10^{16900}$

Function approximation

De tablas a funciones...

estado	valor
s_1	$v(s_1)$ ✓
s_2	$v(s_2)$ ✓
$\vdots$	$\vdots$
s_n	$v(s_n)$ ✓



$\hat{v}(s;w) \approx v(s) \quad \forall s \in S$

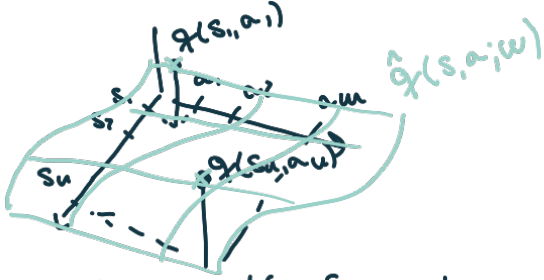
w: parámetros a ajustar

$\hat{v}(s;w) \quad |w| < n$

De tablas a funciones...

	a_1	a_2		a_m
s_1	$q(s_1, a_1)$	$q(s_1, a_2)$	$\dots$	$q(s_1, a_m)$
s_2	$q(s_2, a_1)$	$q(s_2, a_2)$	$\dots$	$q(s_2, a_m)$
$\vdots$	$\vdots$		$\ddots$	$\vdots$
s_n	$q(s_n, a_1)$	$q(s_n, a_2)$	$\dots$	$q(s_n, a_m)$

$n \times m$



$\hat{q}(a, s; w) \approx q(a, s) \quad \forall s \in \mathcal{S}, a \in \mathcal{A}$

$|w| < n \times m$

Algunos tipos de funciones...

función lineal

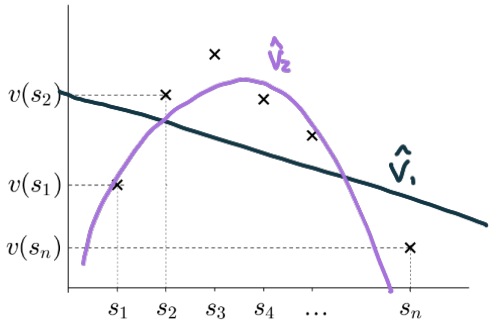
$\hat{v}_1(s;w) = w_1 + w_2 s$

$|w| = 2$

función cuadrática

$\hat{v}_2(s;w) = w_1 + w_2 s + w_3 s^2$

$|w| = 3$



Diremos que la **aproximación es lineal** si se puede escribir de la siguiente forma

$$\hat{v}(s;w) = \sum_{i=1}^{d} w_i \phi_i(s) \qquad \left[= w^\top \phi(s)\right]$$

$w_1 \phi_1(s) + w_2 \phi_2(s) - - + w_d \phi_d(s)$

donde $w = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_d \end{bmatrix}$ y $\phi(s) = \begin{bmatrix} \phi_1(s) \\ \phi_2(s) \\ \vdots \\ \phi_d(s) \end{bmatrix}$

$[w_1\ w_2\ \cdots\ w_d] \begin{bmatrix} \phi_1(s) \\ \phi_2(s) \\ \vdots \\ \phi_d(s) \end{bmatrix}$

Ejemplo: $\hat{v}(s;w) = w_1 + w_2 s + w_3 s^2$ es lineal en w pero no en s.

$$\hat{v}(s;w) = w = \begin{bmatrix} w_1 & w_2 & w_3 \end{bmatrix} \begin{bmatrix} 1 \\ s \\ s^2 \end{bmatrix}$$

$\phi_1(s) = 1$

$\phi_2(s) = s$

$\phi_3(s) = s^2$

Ejemplos no lineales:

1. $\hat{v}(s; w) = w_1 e^{s} + w_2 e^{-s}$ ✗
2. $\hat{v}(s; w) = w_1 + w_2 s + w_1 \cdot w_2$
3. $\hat{v}(s; w) = e^{w_1} s + w_2$
4. $\hat{v}(s; w) = e^{(-(s-w_1)^2)} + e^{(-(s-w_2)^2)}$
5. redes neuronales !

Function approximation: Neural Networks

Neural network

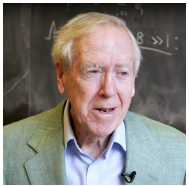
A **fully-connected feedforward neural network** is a function $nn_\theta : \mathbb{R}^{N_1} \to \mathbb{R}^{N_{M+1}}$

$$nn_\theta(x) := L_M \circ L_{M-1} \circ \cdots \circ L_1(x),$$

where $L_i(x) = \sigma_i(W^{[i]}x + b^{[i]})$ is the i-th layer ($L_i : \mathbb{R}^{N_i} \to \mathbb{R}^{N_{i+1}}$).

- σ_i is the i-th **activation function**
- The collection of $W^{[i]}$ and $b^{[i]}$, $i = 1, \ldots, M$ are called **weights** and denoted by θ

G. Strang, The functions of deep learning, *SIAM News*, 2018.



"The best way to create complex functions from simple functions is by composition."

image from https://cse.mit.edu/people/gilbert-strang/

Neural networks can approximate many functions...

Theorem 2. *Let σ be any continuous sigmoidal function. Then finite sums of the form*

$$G(x) = \sum_{j=1}^{N} \alpha_j \sigma(y_j^T x + \theta_j)$$

are dense in $C(I_n)$. In other words, given any $f \in C(I_n)$ and $\varepsilon > 0$, there is a sum, $G(x)$, of the above form, for which

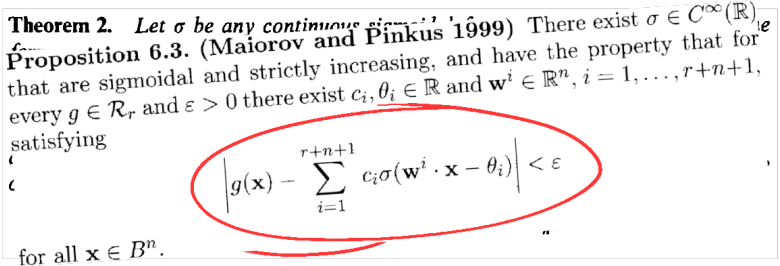
$$|G(x) - f(x)| < \varepsilon \quad \text{for all} \quad x \in I_n.$$

G. Cybenko, Approximation by superposition of a sigmoidal function, *Math. Control Signal Sys.* 1989.

A. Pinkus, Approximation theory of the MLP model in neural networks, *Acta Numer.* 1999.

Z. Lu, et al., The expressive power of neural networks: A view from the width, *NeurIPS*, 2017.

Neural networks can approximate many functions...



Theorem 2. *Let σ be any continuous sigmoidal* (1999) There exist $\sigma \in C^\infty(\mathbb{R})$ *e*

Proposition 6.3. (Maiorov and Pinkus 1999) There exist $\sigma \in C^\infty(\mathbb{R})$ that are sigmoidal and strictly increasing, and have the property that for every $g \in \mathcal{R}_r$ and $\varepsilon > 0$ there exist $c_i, \theta_i \in \mathbb{R}$ and $\mathbf{w}^i \in \mathbb{R}^n$, $i = 1, \ldots, r+n+1$, satisfying

$$\left| g(\mathbf{x}) - \sum_{i=1}^{r+n+1} c_i \sigma(\mathbf{w}^i \cdot \mathbf{x} - \theta_i) \right| < \varepsilon$$

for all $\mathbf{x} \in B^n$.

G. Cybenko, Approximation by superposition of a sigmoidal function, *Math. Control Signal Sys.* 1989.

A. Pinkus, Approximation theory of the MLP model in neural networks, *Acta Numer.* 1999.

Z. Lu, et al., The expressive power of neural networks: A view from the width, *NeurIPS*, 2017.

Neural networks can approximate many functions...

Theorem 2. *Let σ be any continuous sigmoidal* (1999) There exist $\sigma \in C^\infty(\mathbb{R})$ *e*

Proposition 6.3. (Maiorov and Pinkus 1999) *There exist* [illegible] *increasing, and have the property that for*

Theorem 1 (Universal Approximation Theorem for Width-Bounded ReLU Networks). *For any Lebesgue-integrable function $f : \mathbb{R}^n \to \mathbb{R}$ and any $\epsilon > 0$, there exists a fully-connected ReLU network $\mathscr{A}$ with width $d_m \leq n + 4$, such that the function $F_{\mathscr{A}}$ represented by this network satisfies*

$$\int_{\mathbb{R}^n} |f(x) - F_{\mathscr{A}}(x)| \mathrm{d}x < \epsilon. \tag{3}$$

for all $\mathbf{x} \in B^n$.

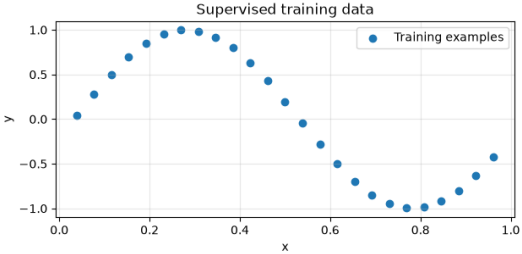
G. Cybenko, Approximation by superposition of a sigmoidal function, *Math. Control Signal Sys.* 1989.

A. Pinkus, Approximation theory of the MLP model in neural networks, *Acta Numer.* 1999.

Z. Lu, et al., The expressive power of neural networks: A view from the width, *NeurIPS*, 2017.

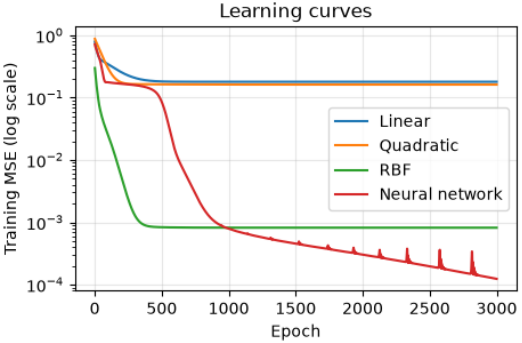
Function approximation: Example

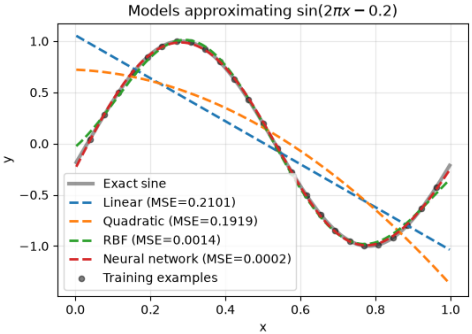
Ver código `Example_01-Function-approximation` y comparar la aproximación obtenida con los siguientes modelos:



- también llamado $\hat{v}_1(x; w) = w_1 + w_2 x$
- $\hat{v}_2(x; w) = w_1 x + w_2 x + w_3 x^2$
- $\hat{v}_3(x; w, c) = w_0 + w_1 \exp\left(-\frac{(x-c_1)^2}{2\sigma^2}\right) + \cdots + w_5 \exp\left(-\frac{(x-c_5)^2}{2\sigma^2}\right)$ RBF
- $\hat{v}_4(x; w) = w_0 + w_{3,i}\sigma(w_{1,1}x + w_{2,1}) + \cdots + w_{3,8}\sigma(w_{1,8}x + w_{2,8})$

Function approximation: Example





Value function methods: Value representation

Objetivo: Usar una función parametrizada (por w) para aproximar v y/o q:

estado	valor
s_1	$v(s_1)$
s_2	$v(s_2)$
$\vdots$	$\vdots$
s_n	$v(s_n)$

$\hat{v}(s;w)$ ← $|w| < u$

parámetros

	a_1	a_2		a_m
s_1	$q(s_1,a_1)$	$q(s_1,a_2)$	$\dots$	$q(s_1,a_m)$
s_2	$q(s_2,a_1)$	$q(s_2,a_2)$	$\dots$	$q(s_2,a_m)$
$\vdots$	$\vdots$		$\ddots$	$\vdots$
s_n	$q(s_n,a_1)$	$q(s_n,a_2)$	$\dots$	$q(s_n,a_m)$

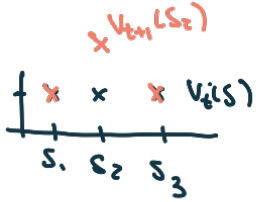
$\hat{q}(s,a;w)$ ← $|w| < u \times u$

Obs: Este enfoque nos da un valor aproximado de la función de valor v_π y de la función de valor de acción q_π.

Value function methods: Value representation

Caso tabular:

$$\begin{cases} v_{t+1}(s_t) = v_t(s_t) - \alpha(s_t)\Big[v_t(s_t) - \big(r_{t+1} + \gamma v_t(s_{t+1})\big)\Big], \\ v_{t+1}(s) = v_t(s), \quad \forall\, s \neq (s_t) \end{cases}$$



Aproximación por función:

$$w_{t+1} = w_t - \alpha\Big[\quad \cdots \quad\Big],$$

$\hat{V}(s;w)$



Value function methods: Objective function

Dada una **política fija** π, definimos $v_\pi(s)$ como la función de valor (exacta) para cada $s \in \mathcal{S}$. Entonces, **queremos encontrar los valores de** w para los cuales $\hat{v}$ es la mejor aproximación de v_π. Para esto, utilizaremos el siguiente funcional:

$$J(w) = \frac{1}{2}\mathbb{E}\left[\left(v_\pi(S) - \hat{v}(S, w)\right)^2\right].$$

$\mathbb{E}(X) = \sum x\, p(x)$

$\mathbb{E}(f(x)) = \sum f(x)\, p(x)$

- El objetivo es encontrar el valor de w que minimiza $J(w)$.
- El valor esperado se calcula con respecto a la variable aleatoria S.
- Necesitamos calcular la distribución de probabilidad de S.

Value function methods: Objective function

- **Distribución uniforme**: asume que cada estado tiene la misma probabilidad $1/n$ de ser elegido (dados n estados).

$$J(w) = \frac{1}{2} \sum_{s \in \mathcal{S}} \frac{1}{n} \left(v_\pi(s) - \hat{v}(s, w) \right)^2$$

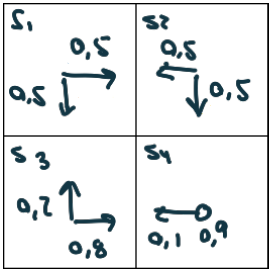
  (no considera la dinámica del proceso de Markov)

- **Distribución estacionaria**: describe el comportamiento a "largo plazo" del proceso de decisión de Markov bajo una política π. En otras palabras, esta distribución, denotada por $d_\pi(s)$, describe la probabilidad de que el agente se encuentre en el estado s después de seguir una política durante un largo periodo.

$$J(w) = \frac{1}{2} \sum_{s \in \mathcal{S}} d_\pi(s) \left(v_\pi(s) - \hat{v}(s, w) \right)^2$$

Value function methods: Objective function

La distribución estacionaria nos da la probabilidad de encontrarnos en un estado después de "infinitos" pasos



$$P_\pi = \begin{array}{c} \\ s_1 \\ s_2 \\ s_3 \\ s_4 \end{array} \begin{array}{c} \begin{array}{cccc} s_1 & s_2 & s_3 & s_4 \end{array} \\ \begin{bmatrix} 0 & 0{,}5 & 0{,}5 & 0 \\ 0{,}5 & 0 & 0 & 0{,}5 \\ 0{,}2 & 0 & 0 & 0{,}8 \\ 0 & 0 & 0{,}1 & 0{,}9 \end{bmatrix} \end{array}$$

$$\begin{array}{cccc} s_1 & s_2 & s_3 & s_4 \end{array}$$

$$[1\ 0\ 0\ 0]\, P_\pi = [0 \quad \overset{s_2}{0{,}5} \quad \overset{s_3}{0{,}5} \quad 0]$$

$$[1\ 0\ 0\ 0]\, P_\pi^2 = [0{,}35 \quad 0 \quad 0 \quad 0{,}65]$$

$$[1\ 0\ 0\ 0]\, P_\pi^3 = [0 \quad 0{,}175 \quad 0{,}24 \quad 0{,}595]$$

$$\vdots$$

$$[0{,}03 \quad 0{,}01 \quad 0{,}1 \quad 0{,}86]$$

Value function methods: Objective function

En el fondo, queremos estudiar qué sucede con

$$P_\pi^k$$

cuando $k \to \infty$ (esta rama de la matemática se conoce como sistemas dinámicos).

La distribución estacionaria se puede calcular resolviendo la ecuación:

$$d_\pi^\top P_\pi = d_\pi^\top$$

d_π ?

(Necesitamos conocer P_π para eso)

---

d_π se puede aproximar de la siguiente forma:

$$d_\pi(s) \approx \frac{n_\pi(s)}{\sum_{s' \in \mathcal{S}} n_\pi(s')}$$

$n_\pi(s)$: número de veces que el estado s es visitado.

Value function methods: Optimization algorithms

Función objetivo:

$$\boxed{J(w) = \frac{1}{2}\mathbb{E}\left[(v_\pi(s) - \hat{v}(s;w))^2\right]}$$

para minimizar J utilizaremos gradient descent:

$$w_{k+1} = w_k - \alpha_k \nabla_w J(w_k)$$

$\nabla_w J(w) =$

Value function methods: Optimization algorithms

El algoritmo de gradient descent en este caso es:

$$\boxed{w_{k+1} = w_k + \alpha_k \mathbb{E}\Big[(v_\pi(S) - \hat{v}(S; w_k))\nabla_w \hat{v}(S; w_k)\Big]}$$

- Esta expresión requiere calcular el valor esperado (también se requiere conocer la distribución de S).
- Podemos reemplazarlo utilizando un gradiente estocástico!
- Reemplazamos k por t para unificar la notación.

$$w_{t+1} = w_t + \alpha_t \Big(v_\pi(s_t) - \hat{v}(s_t; w_t)\Big)\nabla_w \hat{v}(s_t; w_t),$$

donde s_t es una muestra de $\mathcal{S}$ a tiempo t.

Value function methods: Optimization algorithms

$$\boxed{w_{t+1} = w_t + \alpha_t \Big(v_\pi(s_t) - \hat{v}(s_t; w_t) \Big) \nabla_w \hat{v}(s_t; w_t)}$$

Esta expresión todavía tiene un problema ya que depende de v_π

- **Monte Carlo**: Dado un episodio $(s_0, r_1, s_1, r_2, \dots)$, definimos g_t como el retorno (con descuento) a partir de s_t. Entonces g_t se puede usar como aproximación de $v_\pi(s_t)$:

$$w_{t+1} = w_t + \alpha_t \Big(g_t - \hat{v}(s_t; w_t) \Big) \nabla_w \hat{v}(s_t; w_t)$$

- **Temporal-difference**: En el espíritu de TD learning, podemos usar $r_{t+1} + \gamma \hat{v}(s_{t+1}; w_t)$ para aproximar $v_\pi(s_t)$:

$$w_{t+1} = w_t + \alpha_t \Big(r_{t+1} + \gamma \hat{v}(s_{t+1}; w_t) - \hat{v}(s_t; w_t) \Big) \nabla_w \hat{v}(s_t; w_t)$$

Value function methods: Optimization algorithms

Temporal-differences nos permite aproximar la **función de valor** para cada estado s por medio de $\hat{v}(s; w)$:

$$\boxed{w_{t+1} = w_t + \alpha_t \Big(r_{t+1} + \gamma \hat{v}(s_{t+1}; w_t) - \hat{v}(s_t; w_t) \Big) \nabla_w \hat{v}(s_t; w_t)}$$

- En cada paso t requiere $\{(s_t, r_{t+1}, s_{t+1})\}$
- Solamente estima la función de valor para una política dada
- De alguna forma tratamos de resolver la ecuación de Bellman (Bellman expectation equation)

$$v_\pi(s) = \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s]$$

- No estima el valor de las acciones ni calcula políticas óptimas